

ESP32-S3 Pixel Playback — Rebuild Plan (Step-by-Step)

Goal & Scope

Rebuild an ESP32-S3 firmware that streams LED pixel data over Wi-Fi from Art-Net/sACN/KiNet and plays recorded files from SD. The plan uses PlatformIO + Arduino (ESP-IDF where noted), emphasizes deterministic buffering, and adds observability. Each step is a single, testable increment.

0) Project Bootstrap

1. Repository & File Structure

- **Create repo skeleton**
- `platformio.ini` — board: esp32-s3 (adjust to your module), upload_speed, monitor_speed, PSRAM, partitions.
- `src/main.cpp` — minimal setup/loop; Serial prints only.
- `include/config.h` — compile-time defaults (Wi-Fi SSID, pins, LED count).
- `lib/` — placeholder subfolders: `LEDManager/`, `Protocols/` (ArtNet, sACN, KiNet, Common), `SDCard/`, `Playback/`, `WebUI/`, `Utils/`.
- `data/` — web UI assets (served by SPIFFS/LittleFS).
- `test/` — for unity tests (optional).

platformio.ini (starter)

```
[env:esp32s3]
platform = espressif32
board = esp32s3dev
framework = arduino
board_build.arduino.memory_type = qspi_opi
build_flags =
    -DCORE_DEBUG_LEVEL=3
    -DBOARD_HAS_PSRAM
monitor_speed = 115200
upload_speed = 921600
board_build.partitions = default.csv
lib_deps =
    fastled/FastLED
    me-no-dev/ESP Async WebServer
    bblanchon/ArduinoJson
```

2. Serial Link & Logging

- Open Serial at **115200**; print boot header (chip, flash, PSRAM availability).
- Add ASSERT/VERIFY macros and `LOGx()` helpers (tagged levels).
- Verify correct COM/tty and baud via `pio device monitor`.

`src/main.cpp` (smoke test)

```
void setup(){
  Serial.begin(115200);
  delay(200);
  Serial.println("Boot: ESP32-S3 Pixel Playback");
}
void loop(){ delay(100); }
```

3. Wi-Fi Bring-Up

- Create `lib/WiFiManager/` with STA connect + fallback AP (captive portal optional).
- JSON status: IP, RSSI, hostname; retries with backoff; persist SSID/pwd in Preferences.
- *Acceptance*: ping from host; confirm stable IP and RSSI logging.

4. Task Layout & Core Pinning

- Define RTOS tasks:
- **rxTask** (protocol packets) on **APP** CPU,
- **renderTask** (LED) on **PRO** CPU,
- **ioTask** (SD).
- Pre-allocate queues/ring buffers at boot; no dynamic `malloc` after setup.
- Add `/api/stats` to dump queue depths, drops, FPS.

Networking & Protocols

5. Protocol Abstraction

- Create `IProtocolAdapter` with `begin(iface)`, `join(universe set)`, `onPacket(cb)`, `stats()`.
- Implement `DMXUniverseAssembler` to gather 512-ch frames by (**universe, sequence**).
- Counters: packets, late, missing, dupes, last sequence.

6. Art-Net Adapter (UDP 6454)

- Use **lwIP raw UDP**; register RX callback; copy payload into RX ring node (1536 B).
- Parse **OpDmx**; map to universe; push to assembler; emit frame-ready events.
- Unit test with a small PC sender; verify PPS and loss metrics.

7. sACN/E1.31 Adapter

- Join multicast for selected universes; parse PDU layers; track per-source **CID**.
- Honor per-frame sequence; add **frame fence timeout** (5–8 ms) before render.
- Optional: implement **sACN Synchronization** packets for lockstep.

8. KiNet Adapter

- Listen on KiNet port; parse v1/v2 as needed; map to assembler pipeline.
- Share common RX ring and stats.

Networking I/O tuning - Per-protocol **RX ring buffers** (single-producer/single-consumer). - Drop policy on overflow; expose counters. - Increase lwIP receive buffers; pin tasks as above. - Optional DSCP/ `IP_TOS` marking for low latency.

LED Output Pipeline

9. LED Manager & Mapping

- Define pixel format (RGB/RGBW, order) and total pixel count in `include/config.h`.
- Precompute mapping from **(universe, channel) → framebuffer offset**.
- **Double buffer**: front (scanned) + back (filled). Consider triple buffer when SD + network both active.

10. LED Driver

- **WS281x**: RMT with DMA (or FastLED if adequate) at stable refresh period.
- **APA102/SK9822**: SPI DMA, high clock, packed bytes; measure scan time vs FPS target.
- Add **gamma LUT** + optional **temporal dithering** (toggle).

11. Render Scheduler

- `esp_timer` periodic tick at target FPS; swap buffers if frame-ready; else reuse last.
 - Stats: render FPS, overrun count, avg scan μ s.
-

SD Playback & File Format

12. SD Interface

- Prefer **SDMMC 4-bit**; else SPI @ 40–80 MHz on dedicated bus; enable DMA.
- Implement async reader task with **128–512 KB** read-ahead ring.

13. Recording File Spec (v1)

- Header: magic, version, fps, pixel_count, bpp, universe_map, flags.
- Per-frame chunk: `[timestamp | frame_size | payload | crc32]`.
- Index block every N frames for seek; store endianness.

14. Playback Engine

- Reader fills ring; scheduler dequeues at recorded timestamps (drift-aware).
 - Backpressure: **pause** on underrun; **resume** when buffer $\geq$ threshold.
-

Web UI & Ops

15. Web Server & APIs

- AsyncWebServer with JSON endpoints:
- `/api/config`, `/api/protocol`, `/api/stats`,
- `/api/playback` (open/close/play/pause/seek).
- Serve SPA from `data/`; add simple auth (token).

16. Observability & Reliability

- Watchdog for render and RX tasks; graceful adapter restart on fault.
- Circular log buffer via `/api/logs`; counters for drops/late/underrun.

17. Milestone Checklist

- **[M1]** Serial online; logging prints.
 - **[M2]** Wi-Fi STA stable; REST `/api/stats` returns Wi-Fi info.
 - **[M3]** Art-Net RX → LED test pattern replace; show packet counters.
 - **[M4]** sACN RX with frame fence; verify low jitter.
 - **[M5]** LED double-buffer render at 40–60 FPS.
 - **[M6]** SD read-ahead throughput; dummy playback frames.
 - **[M7]** File v1 playback end-to-end with seek.
 - **[M8]** Web UI controlling protocol & playback; live stats.
 - **[M9]** Long-run soak test (2–4 h) with jitter/drops < target.
-

Best-Practice Notes

- Allocate large buffers at boot; minimize heap fragmentation; prefer PSRAM for big rings.
- Provide **preset profiles** (Low-Latency / High-Fidelity / Playback-Priority) that toggle internal tunables.
- Backpressure semantics: for **live**, drop-to-latest; for **playback**, pause on underrun.
- Add built-in packet/test pattern generator for offline verification.